Linux Command Line Fundamentals Lab Study Guide

How to use this guide: Read the section for your current module *before* attempting the practical challenges. Use the command tables and deep-dive conceptual sections as a reference. The examples provided here use fictional paths and files (e.g., `/var/backup/syslog`, `doc_archive`, `worker_script.sh`) to build your terminal skills without spoiling the exact answers to the lab objectives. Always check the manual pages (`man`) when you need to understand specific tool options.

Table of Contents

Module 1: Linux Navigation & Filesystem Exploration

Relevant Module: Module 1

Module 1: Conceptual Background

In graphical operating systems, users browse folders and click icons to open files. In server administration and DevOps, servers usually run without a graphical interface to save resources. Sysadmins must interact with the system using a **Command Line Interface (CLI)**.

At the core of Linux navigation is the **hierarchical directory tree**. Every folder is a branch, starting from the root directory (`/`). Navigating this structure quickly is the first step to performing diagnostic or configuration tasks on a Linux host.

Module 1: Key Commands Reference

Command	Purpose	Key Flags	Example
`pwd`	Print Working Directory (shows exactly where you are)	—	`pwd`
`ls`	List directory contents	`-a` show hidden files; `-l` long format list	`ls -la`
`cd`	Change directory (move to a new path)	—	`cd /var/log`
`cat`	Concatenate and display file content	—	`cat /etc/hostname`

Module 1: Core Concepts Deep-Dive

Absolute vs. Relative Paths

When changing directories or opening files, you can specify their location in two ways:

1. **Absolute Path**: A path that starts from the root (`/`). It is unambiguous and points to the same file regardless of your current directory (e.g., `cd /var/log/nginx`).
2. **Relative Path**: A path relative to your current location. It does not start with a slash (e.g., if you are in `/var`, typing `cd log` moves you to `/var/log`).

Special Directory References

- `.` (single dot): Refers to the **current directory**.
- `..` (double dot): Refers to the **parent directory** (one level up).
- `~` (tilde): Refers to the current user's **home directory** (e.g., `/home/student`).

Hidden Files (Dotfiles)

In Linux, any file or directory whose name starts with a dot (`.`) is treated as hidden. A standard `ls` command will not show it. To view these files, you must run `ls -a`. These are often used for user configurations, settings, or credentials.

Module 1: Worked Examples

Example: Locating a hidden key in a nested tree

Situation: You need to find a secret configuration file hidden inside a nested folder structure under your home directory.

```
# Verify current location
pwd

# Output: /home/testuser

# List contents (including hidden ones)
ls -la
# Output:
# drwxr-xr-x 3 testuser testuser 4096 Aug  5 10:00 .
# drwxr-xr-x 3 root     root     4096 Aug  5 10:00 ..
# drwxr-xr-x 3 testuser testuser 4096 Aug  5 10:00 archive_folder
```

```
# Move into the nested archive folder
cd archive_folder/configs/keys

# Check directory contents (standard ls shows nothing)
ls

# Check with -a flag to list hidden files
ls -a

# Output: .  ..  .secrets.conf

# Read the hidden configuration file
cat .secrets.conf

# Output: API_TOKEN=FLAG{dummy_nav_flag_123}
```

Module 1: Before You Start This Module

1. What command tells you the absolute path of the directory you are currently working in?
2. If your shell is currently inside `/var/log` and you run `cd ..`, where will you be?
3. How can you list a directory's contents to verify if it contains hidden files?

Module 1: Common Mistakes

- **Confusing `/` and `~`.** `/` takes you to the system root, whereas `~` takes you to your user's home folder.
- **Type errors with relative paths.** Typing `cd /records` instead of `cd records` (without the leading slash) will look for `records` in the system root rather than your current directory, causing a "No such file or directory" error.

Module 1: Further Reading

- `man pwd` — Detail specifications for the print working directory utility.
- `man ls` — Documentation of all listing options, including file sizes and sorting flags.
- `man cd` — Reference documentation for shell built-in directory navigation.

Module 2: File Operations & Workspace Management

Relevant Module: Module 2

Module 2: Conceptual Background

Managing files, creating directories, copying configurations, renaming scripts, and deleting obsolete temporary files, is the daily routine of a systems engineer.

Linux treats folders and files with a simple set of commands. Because there is no trash bin in the command line, operations like `rm` are permanent. Executing these operations precisely ensures your workspaces are clean and configuration assets are not accidentally lost or overwritten.

Module 2: Key Commands Reference

Command	Purpose	Key Flags	Example
`mkdir`	Make a directory	`-p` create nested parent directories as needed	`mkdir -p project/backup`
`cp`	Copy files or directories	`-r` copy recursively (required for folders)	`cp config.txt config.bak`
`mv`	Move or rename files and directories	—	`mv draft.txt final.txt`
`rm`	Remove (delete) files	`-r` recursive delete; `-f` force delete	`rm junk.tmp`

Module 2: Core Concepts Deep-Dive

Copying vs. Moving

- **`cp` (Copy)**: Duplicates a file. The original file remains in place, and a new identical file is created at the destination.
- **`mv` (Move)**: Relocates a file. The original file is removed from its source directory and placed in the target directory. `mv` is also the command used to rename files.

Dangerous Options: `rm -rf`

The `-r` (recursive) flag deletes directories and their contents. The `-f` (force) flag overrides prompts and ignores non-existent files. Combining these as `rm -rf` makes the command completely silent and permanent. Running it on the wrong path can break your operating system or destroy valuable data.

Module 2: Worked Examples

Example: Organizing a dev workspace

Situation: You want to backup a developer draft configuration, move the active draft to a deployment file, and clean up temporary files in your workspace.

```
# Create directory structure
mkdir -p dev_workspace/backup

# Copy the original config template to backup folder
cp draft_template.conf dev_workspace/backup/

# Rename/move the draft to final location
mv draft_template.conf dev_workspace/final.conf

# Delete temporary log files
rm temp_debug.log
```

Module 2: Before You Start This Module

1. How do you rename a file from `oldname.txt` to `newname.txt` in the terminal?

2. What flag must be passed to `mkdir` to create nested folders (e.g. `a/b/c`) in a single command?
3. What is the key difference between the `cp` and `mv` commands?

Module 2: Common Mistakes

- **Accidental overwrite with `cp` or `mv`.** If the destination filename already exists, `cp` and `mv` will overwrite it without warning.
- **Forgetting `-r` when copying folders.** Running `cp folder1/ folder2/` will fail with an error stating that the directory is omitted unless the recursive `-r` flag is specified.

Module 2: Further Reading

- `man cp` — Reference manual for file copying parameters.
- `man mv` — Options and usage guide for moving/renaming.
- `man rm` — Safe usage practices and options.

Module 3: Searching & Filtering Log Files

Relevant Module: Module 3

Module 3: Conceptual Background

Modern Linux servers produce massive amounts of logs like, auditing records, database events, application exceptions, and web server logs. When diagnosing an incident, a sysadmin cannot open and read million-line log files manually.

Linux provides powerful text-processing tools that allow administrators to search through directories, filter files based on pattern matching, and view files in real time as events occur.

Module 3: Key Commands Reference

Command	Purpose	Key Flags	Example
`find`	Search the directory tree for files matching criteria	`-type f` find files; `-name` search by name	`find /var/log -name "*.log"`
`grep`	Print lines matching a text pattern	`-r` recursive search; `-i` ignore case	`grep -ri "error" /var/log/`
`head`	Output the first part of files	`-n N` show first N lines	`head -n 10 syslog.log`
`tail`	Output the last part of files	`-n N` show last N lines; `-f` follow changes live	`tail -f nginx.log`

Module 3: Core Concepts Deep-Dive

Recursive Grep

If you know a keyword (like a service name or credential token) exists somewhere in a directory but do not know which file holds it, you can run a recursive search:

```
grep -r "ACCESS_TOKEN" /opt/app/configs/
```

This command reads every file inside `/opt/app/configs/` and its subdirectories, printing any lines containing the matching text along with the file path.

Monitoring Logs with Tail

The `tail` command is used to inspect the end of a file. Using the `-f` (follow) flag keeps the file open in your terminal, printing new log entries as they are written to disk by application processes.

Module 3: Worked Examples

Example: Finding a leaked string in log archives

Situation: An engineer reported that an access token was logged in one of the application log files in `log_archive/`. You need to find which file contains it and inspect its surrounding lines.

```
# Locate all files in the archive to check structure
find log_archive/ -type f

# Output:
# log_archive/2026-05/service_audit.log
# log_archive/2026-06/syslog.log

# Search recursively for the string "TOKEN"
grep -r "TOKEN" log_archive/

# Output:
# log_archive/2026-06/syslog.log:TOKEN: FLAG{search_and_filter_key_999

# View the final 5 lines of the target log file
tail -n 5 log_archive/2026-06/syslog.log
```

Module 3: Before You Start This Module

1. What command searches for files by name in a directory tree?
2. How do you search for a specific string inside multiple files recursively?
3. Which tool and flag allow you to view a log file updating in real-time?

Module 3: Common Mistakes

- **Case sensitivity in `grep`.** Running `grep "error"` will not match `ERROR` or `Error`. Always use the `-i` flag to ignore case if you are unsure of the exact format.

Module 3: Further Reading

- `man grep` — In-depth guide on matching patterns, regular expressions, and output options.
- `man find` — Detailed reference on locating files by size, date, user, or permissions.
- `man tail` — Documentation on monitoring and file slicing commands.

Module 4: Permissions & Basic Shell Scripting

Relevant Module: Module 4

Module 4: Conceptual Background

Linux enforces system security by specifying who can read, modify, or execute files. Executing a script requires the system to check two conditions:

1. Does the script have **execute permission** set?
2. Does the running user have **read permission** to load the script content, and **ownership** of any output logs?

Additionally, sysadmins write simple shell scripts to automate checks. To perform administrative tasks, users escalate their access temporarily using `sudo`.

Module 4: Key Commands Reference

Command	Purpose	Key Flags	Example
`chmod`	Change file permission flags	`+x` add execute permission	`chmod +x run.sh`
`chown`	Change file owner	—	`sudo chown student log.txt`
`sudo`	Execute a command as the superuser (root)	—	`sudo chown student report.csv`

Module 4: Core Concepts Deep-Dive

Execution Permissions (`+x`)

When you write a bash script, it is just a plain text file. The operating system will refuse to execute it directly (e.g., `./script.sh` will return "Permission denied") unless the execute bit (`x`) is added to the file's metadata.

```
chmod +x script.sh
```

Administrative Privilege Escalation (`sudo`)

Only the owner of a file (or the root administrator) can change its ownership. If a file is owned by `root` or another service, you cannot run `chown` to make yourself the owner unless you prefix the command with `sudo` to run it with administrative privileges:

```
sudo chown user_name target_file.txt
```

Module 4: Worked Examples

Example: Preparing and executing an admin utility script

Situation: You have a cleanup script `cleanup_utility.sh` that needs execute permissions, and it needs to access a report file currently owned by `root`.

```
# Add execute permissions to the script
chmod +x cleanup_utility.sh

# Change ownership of the target file so the script can read it
sudo chown student data_report.txt

# Run the script from the current directory
./cleanup_utility.sh
```

Module 4: Before You Start This Module

1. What command prefix allows a standard user to run commands with administrative privileges?

2. If you try to run `./test.sh` and receive a "Permission denied" error, how do you resolve it?
3. How do you change the owner of a file to user `student`?

Module 4: Common Mistakes

- **Forgetting `./` when running a script.** Linux does not search the current directory for executable files by default. Typing `run.sh` will cause a "command not found" error. You must explicitly specify the path to the current directory: `./run.sh`.
- **Forgetting `sudo` when modifying system files.** Changing ownership of files owned by root will fail with a "Operation not permitted" error unless `sudo` is used.

Module 4: Further Reading

- `man chmod` — Comprehensive manual for file permissions.
- `man chown` — Reference documentation for changing owner and group metadata.
- `man sudo` — Secure administrative privilege control guide.

Module 5: Process Control & Stream Redirection (Capstone)

Relevant Module: Module 5

Module 5: Conceptual Background

The Linux capstone requires managing background tasks and manipulating data streams. Every program running on Linux is a **process** identified by a unique Process ID (PID). If a process gets stuck or runs out of control, you must locate its PID and terminate it.

Furthermore, Linux commands do not have to run in isolation. Using **redirection** and **pipes**, you can capture command outputs, pass them to other tools, and write the final output directly to files on disk.

Module 5: Key Commands Reference

Command	Purpose	Key Flags	Example
`ps`	Report snapshot of current processes	`aux` shows all processes running on the system	`ps aux`
`kill`	Terminate a process by sending a signal	`-9` force kill (immediate halt)	`kill -9 1234`
`awk`	Text scanning and processing language	Print specific columns from stdout	`awk '{print $1}'`
`>`	Redirect standard output to a file (overwrites file)	—	`echo "data" > file.txt`
`	`	` (pipe)	Pass output of command A as input to command B	—

Module 5: Core Concepts Deep-Dive

Finding and Killing Runaway Processes

If a process is consuming resources or blocking a port, you can list running processes to find its PID:

```
ps aux | grep sleep
```

This searches the running process list for any processes with "sleep" in their name. The PID is listed in the second column. Once found, terminate it using:

```
kill PID_NUMBER
```

Stream Redirection and Pipe Manipulation

- `>` **(Output Redirection)**: Writes output to a file. E.g., `command > file.txt` saves the output, overwriting any previous content.
- `|` **(Pipe)**: Connects commands. E.g., `cat list.txt | awk '{print $2}'` takes the content of `list.txt`, passes it to `awk`, which extracts only the second column of text.

Module 5: Worked Examples

Example: Stopping a task and extracting field data

Situation: Locate a running process called `dummy_job`, terminate it, read a key file, and extract only the first column of the file to a new file named `out.txt`.

```
# Locate the runaway process PID
ps aux | grep dummy_job

# Output:
# student   4567  0.0  0.0   9000  sleep 1000

# Terminate the process by PID
kill 4567

# Extract the first column from log.txt and write to output file
```

```
cat log.txt | awk '{print $1}' > out.txt

# Verify the output
cat out.txt
```

Module 5: Before You Start This Module

1. How do you find the PID of a running process named `service_worker`?
2. What command terminates a process once you know its PID?
3. What operator redirects command output to overwrite a file on disk?

Module 5: Common Mistakes

- **Forgetting that `kill` takes a PID, not a process name.** Running `kill dummy_job` will fail. You must find the numeric PID (e.g. `4567`) first, then run `kill 4567`.
- **Using `>` instead of `>>` when you want to append.** `>` overwrites the target file completely. If you want to keep the existing content and add to it, use `>>` instead.

Module 5: Further Reading

- `man ps` — Guide on process reporting and display formatting.
- `man kill` — List of termination signals (SIGTERM, SIGKILL).
- `man awk` — Text pattern scanning and processing reference manual.
